

Documentazione CampoService

Scopo della classe

CampoService gestisce la logica di business relativa all'entità **Campo**. Estende **AbstractService<Campo, CampoDto>**, ereditando le operazioni CRUD generiche e aggiungendo metodi di ricerca specifici per la gestione delle strutture sportive.

Dipendenze

- **Logger**: registra informazioni ed errori durante l'esecuzione del codice.
- **CampoMapper**: converte Entity **Campo** in **CampoDto** e viceversa.
- **CampoRepository**: comunica con il database tramite JPA e contiene le query personalizzate relative ai campi.

Costruttore

Riceve tramite Dependency Injection il repository generico, il converter, il mapper e il repository specifico (**CampoRepository**). Il costruttore richiama **super(repository, converter)** per inizializzare la superclasse e salva le dipendenze locali.

Metodi

findByNome(String nome)

Ricerca una struttura o campo sportivo tramite il suo nome esatto. Il repository esegue la query di ricerca sulla base dati, il mapper converte l'Entity trovata nel corrispondente **CampoDto** e il metodo lo restituisce al chiamante.

findByTipologia(String tipologia)

Restituisce tutti i campi appartenenti a una determinata categoria o tipologia (es. "Tennis", "Calcetto"). Recupera la lista di entità dal database e le trasforma in una lista di DTO tramite **CampoMapper**.

findByCoperto(Boolean coperto)

Filtra tutti i campi presenti a sistema in base alla presenza o meno della copertura (campo coperto o al coperto/scoperto). Il risultato viene convertito e restituito come lista di **CampoDto**.

findByTipologiaAndCoperto(String tipologia, Boolean coperto)

Combina due filtri di ricerca simultanei: tipologia del campo e presenza della copertura. Interroga il database per estrarre le sole strutture che soddisfano entrambi i criteri e ne restituisce la lista convertita in DTO.

existsByNome(String nome)

Verifica l'esistenza nel database di un campo con il nome indicato. Restituisce un valore booleano (**true** se il campo esiste già, **false** altrimenti). Questo metodo è fondamentale per eseguire controlli di validazione ed evitare la creazione di duplicati prima dell'inserimento.

Flusso di esecuzione

Controller → CampoService → CampoRepository → Database → Entity Campo → CampoMapper → CampoDto → Controller → JSON.

Vantaggi

Separazione delle responsabilità, riutilizzo del codice CRUD tramite **AbstractService**, conversione Entity/DTO centralizzata, codice pulito, sicuro e facilmente manutenibile.